

Chapter 2. Actors, Humans, and How We Live

Imagine a world where most people are glued to small, hand-held devices that let them send messages to other humans across oceans and continents... wait, we already live in this world!

With actors, it is much the same. The only way to contact a software actor is to send it a message, much like how we exchange text messages on mobile devices. As an example, consider a typical text message exchange between you and a friend. While commuting to work you text your friend and say “Good morning” (see [Figure 2-1](#)).

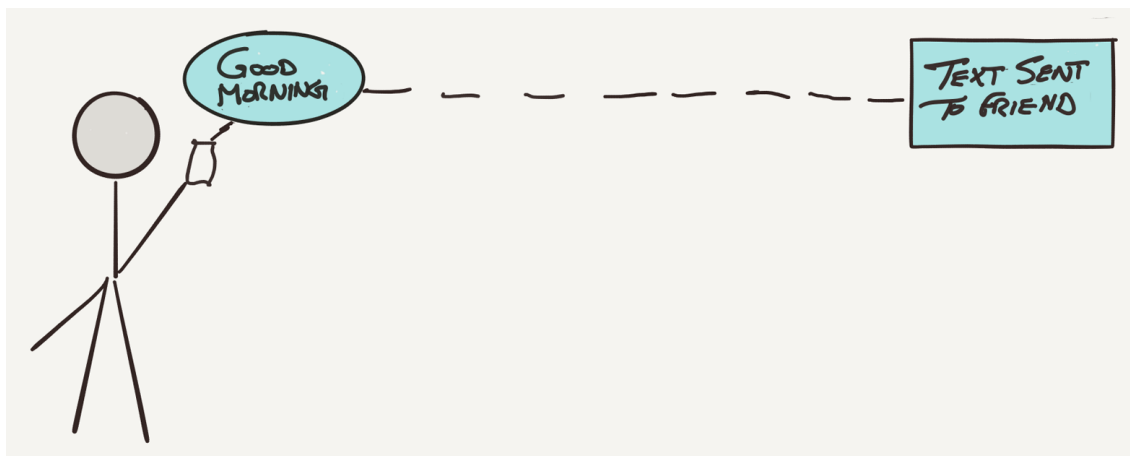


Figure 2-1. Actor messages are like text messages

After you send your friend a message and before she responds, you are free to do other things, such as sending text messages to other friends. It’s conceivable that you would also receive requests via text messages to perform other tasks, which may send you off to do other things and interact with other people.

Your friend may quickly see the message, and responds “Hello, how R U today?” (see [Figure 2-2](#)).

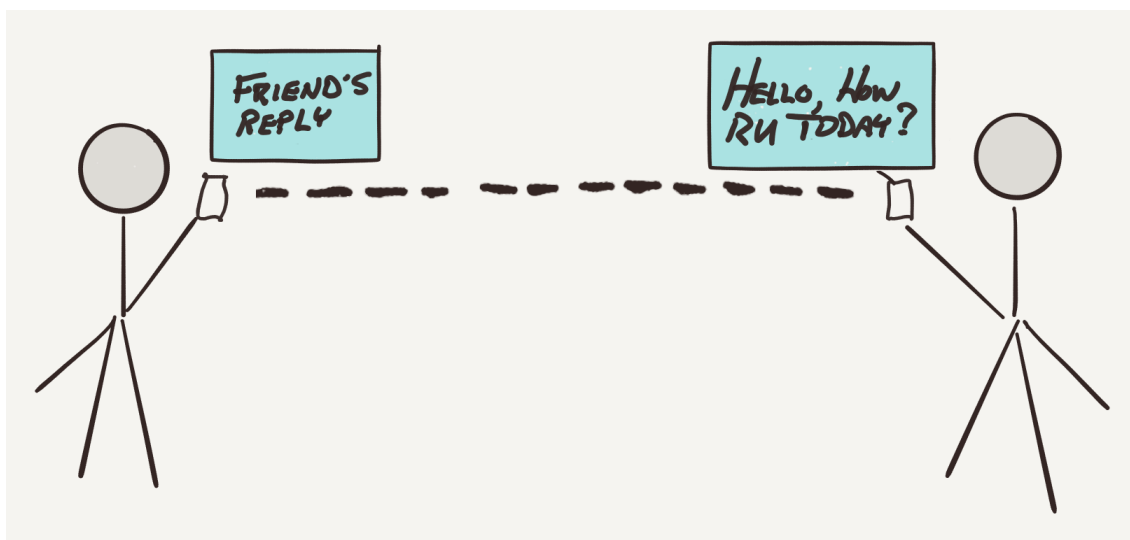


Figure 2-2. Actors behave like humans exchanging text messages

This is basically how messages between software actors behave. When an actor sends a message to another actor, it does not wait for a response; it is free to do other things, such as send messages to other actors.

When you send a text message to a friend or colleague there are a number of possible outcomes. The typical expected outcome is that a short time later you get a response message from your friend. Another possible outcome is that you never get a response to your message.

If you expect a response and never get one (see [Figure 2-3](#)), you might wait for a while and then try to send her another text message (see [Figure 2-4](#)).

In between getting a quick response and no response is the possibility of getting a response after you are unable to or uninterested in continuing the conversation, because your attention is focused elsewhere. In this example texting conversation scenario, you still do not get a response.

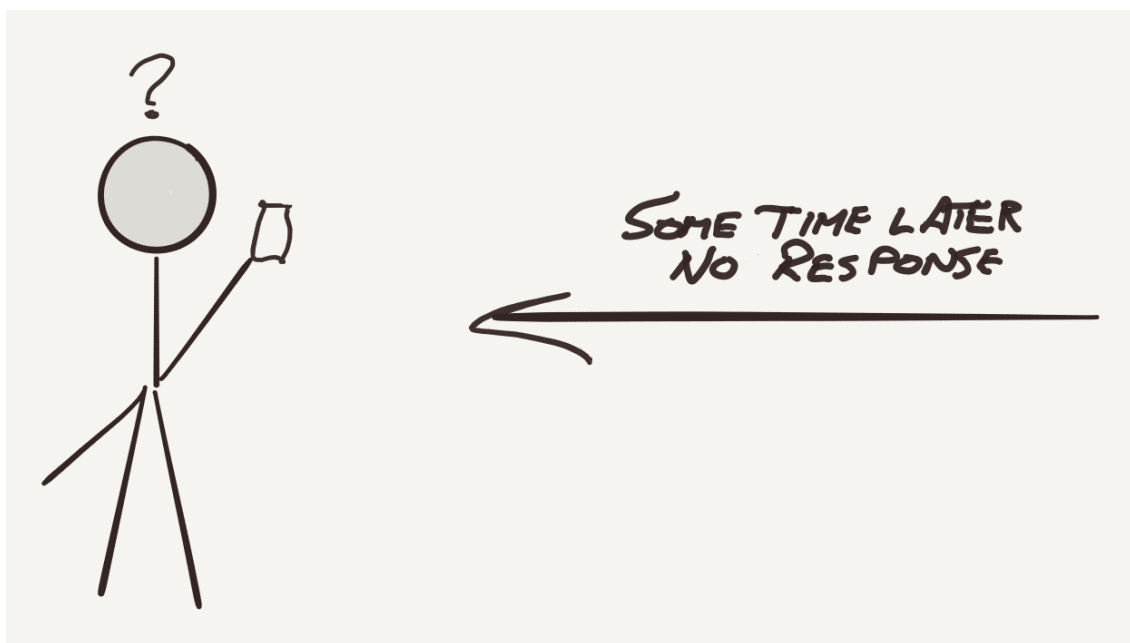


Figure 2-3. No response to your text, what do you do?

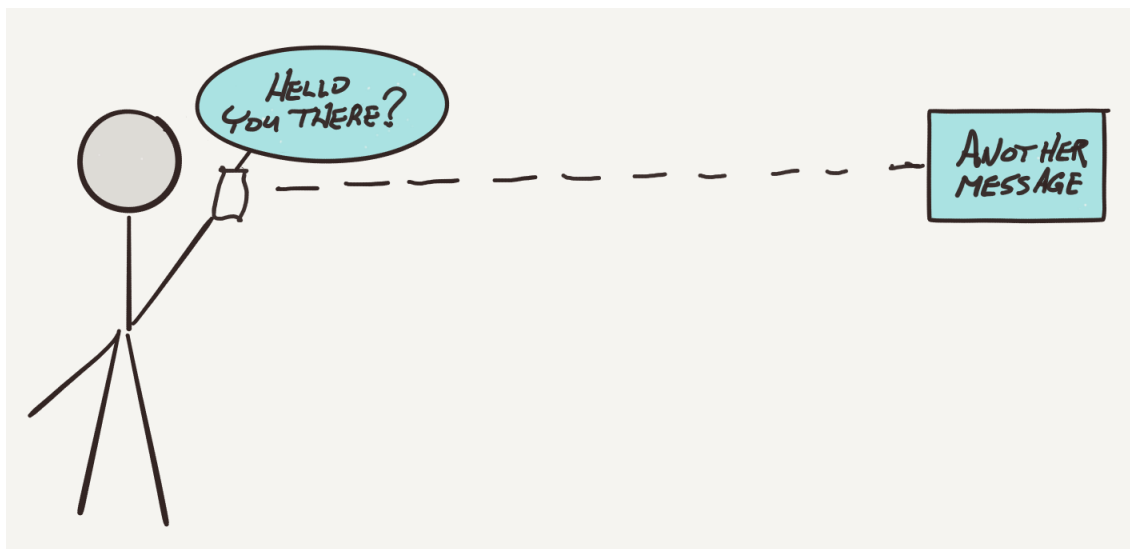


Figure 2-4. No response to a text, send another text

At this point you may be getting somewhat concerned (see [Figure 2-5](#)). You expected your friend to respond, and now you are unable to get in touch with her. So what do you do next? This is where you may have to consider your options, such as:

- Wait some more and try to message her again
- Call your friend by phone
- Get in touch with someone else that may know the status of your friend

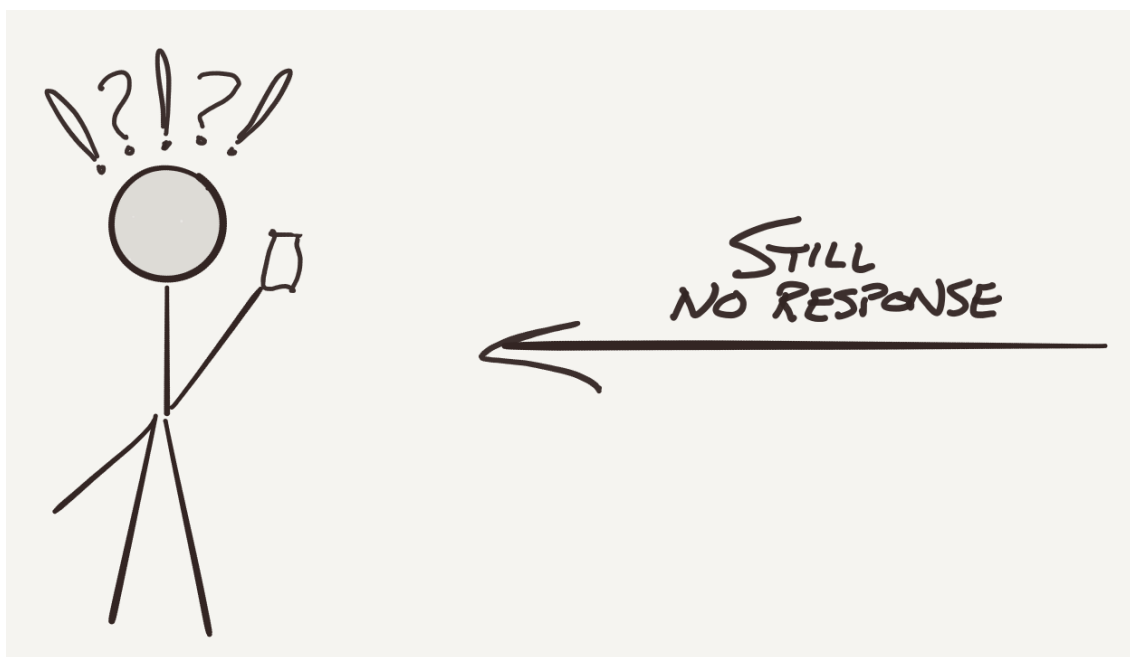


Figure 2-5. Still no response

To be fair, there is a fourth option: stand still and do nothing until you hear from your friend. Blocking all activities until a response is received is probably not considered a feasible thing for most humans, and would make for a pretty inefficient world; yet in many traditional systems, this is precisely what happens, and why the Actor model was created.

Just as with humans sending text messages, actors that can send and receive messages are prepared for multiple possible outcomes: a response may be received quickly, there may be no response at all, or there may be a response that is too late to be useful. The point here is that the actor may be implemented to handle one or more of these possible outcomes.

A well-defined actor can not only handle a typical expected message exchange, it may be capable of handling the other possible outcomes. For example, Actor A sends a message to Actor B. The message is a request to B to perform a specific task.

The desired outcome is for B to perform the task and then send a response message back to A (see [Figure 2-6](#)). There is a possibility that B is unable to perform the request task and, as a result, A never gets a response (see [Figure 2-7](#)). A is prepared for this and has a plan to handle not getting a response.

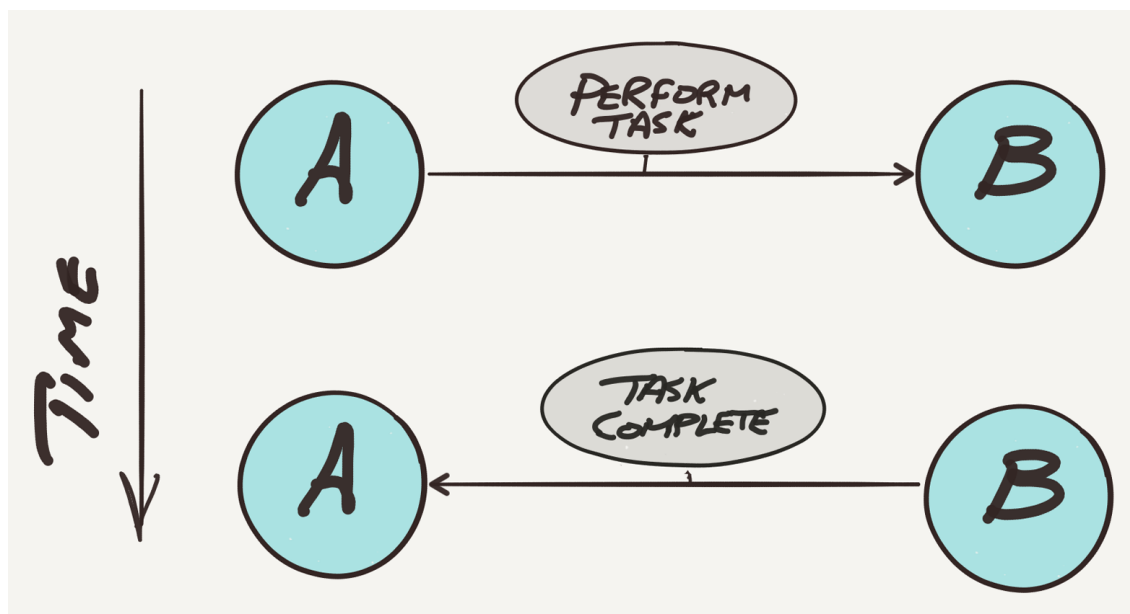


Figure 2-6. Actor A sends a message to B and later gets the expected response

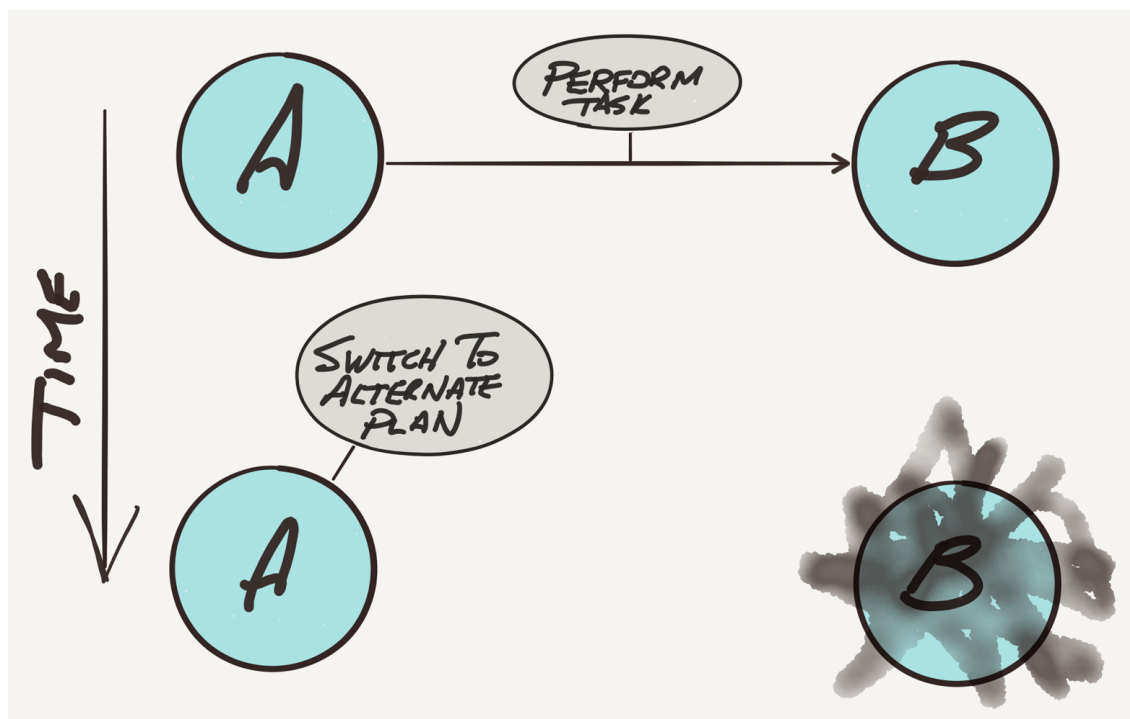


Figure 2-7. Actor B never responds to message from A

A common approach for handling a no-response scenario is for the actor to send a request to another actor to perform a task, setting up a timeout message to be sent to itself at some point in the future. If A asks B to perform a task and B responds back to A before the timeout message arrives, then all is well and A cancels the timeout message (see [Figure 2-8](#)).

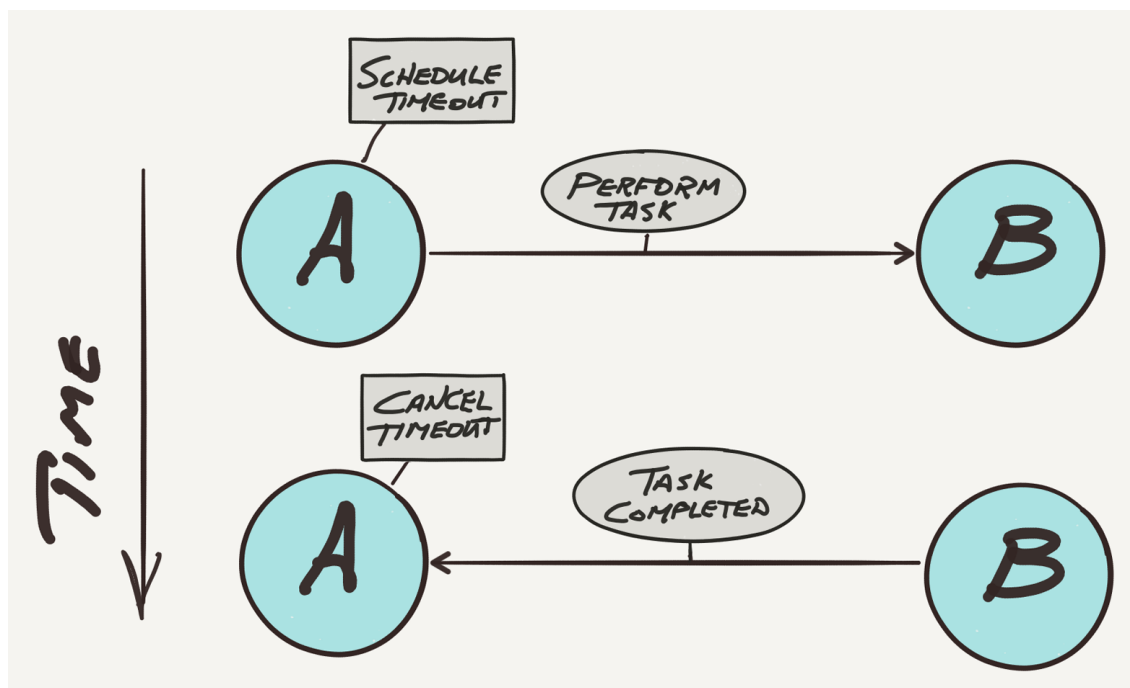


Figure 2-8. Actor A sends message to B and it responds before the timeout

On the other hand, if the timeout message arrives, this triggers A to switch to an alternate plan. How it handles this alternate scenario is very specific to each actor. In some cases, Actor A may simply resend the message to Actor B. In other cases, Actor A may give up and report an error back upstream to the sender of the message that triggered A in the first place (see [Figure 2-9](#)).

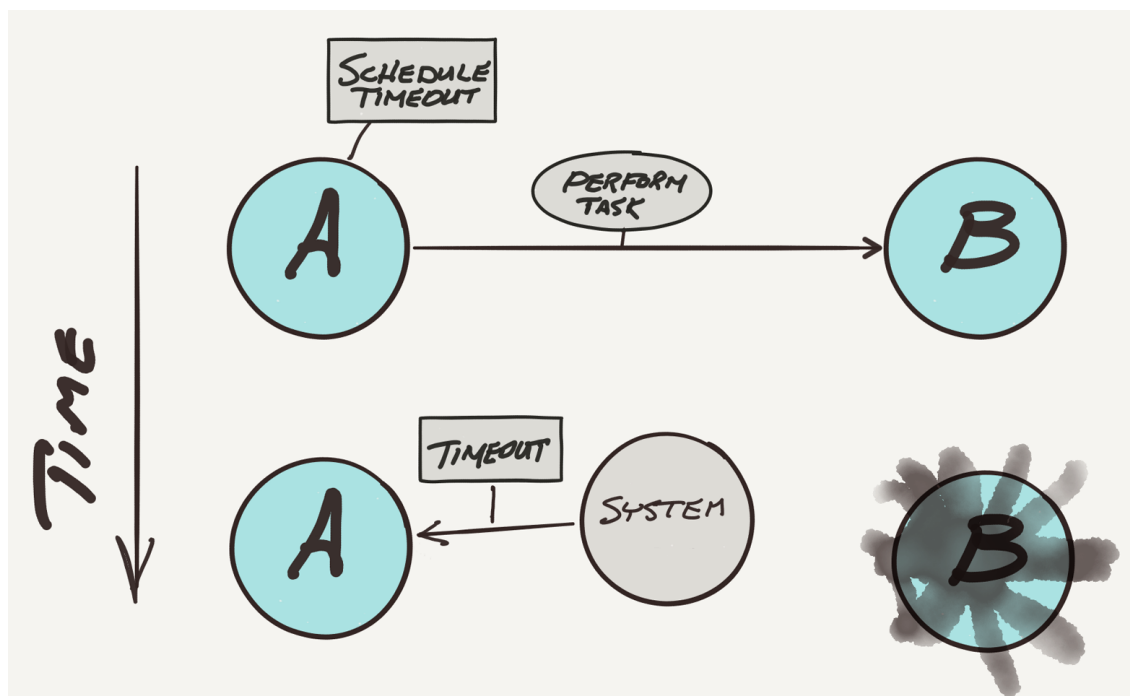


Figure 2-9. No response from Actor B and A gets timeout message

The key point here is that actors exchange messages with each other to communicate. These messages are sent asynchronously, very much in the way humans exchange text messages. Due to this asynchronous behavior,

actors are designed not only for the expected happy path, they are also designed and implemented to handle the unhappy path. In the Actor model, failure handling and recovery is an architectural feature, and not treated as an afterthought.

Actor Supervisors and Workers

Actors may create other actors. When one actor creates another actor, the creator is known as the *supervisor* and the created actor is known as the *worker*. Worker actors may be created for many reasons, but among the most common reasons is for delegating work. The supervisor creates one or more worker actors and delegates work to them (see [Figure 2-10](#)).

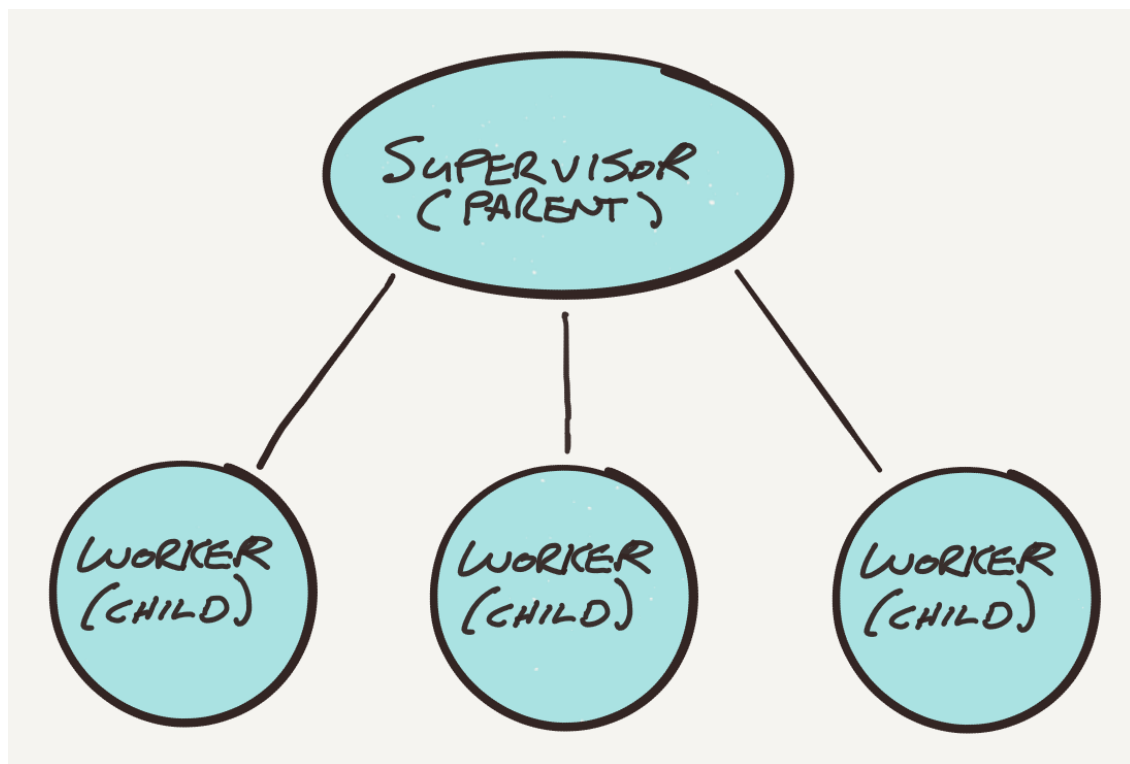


Figure 2-10. Supervisor actor creates worker actors

The supervisor also becomes a caretaker of the workers. Just as with a parent watching over the well-being of their children, the supervisor watches out for the well-being of its workers. If a worker runs into a problem, it suspends itself and notifies its supervisor of the failure ([Figure 2-11](#)).

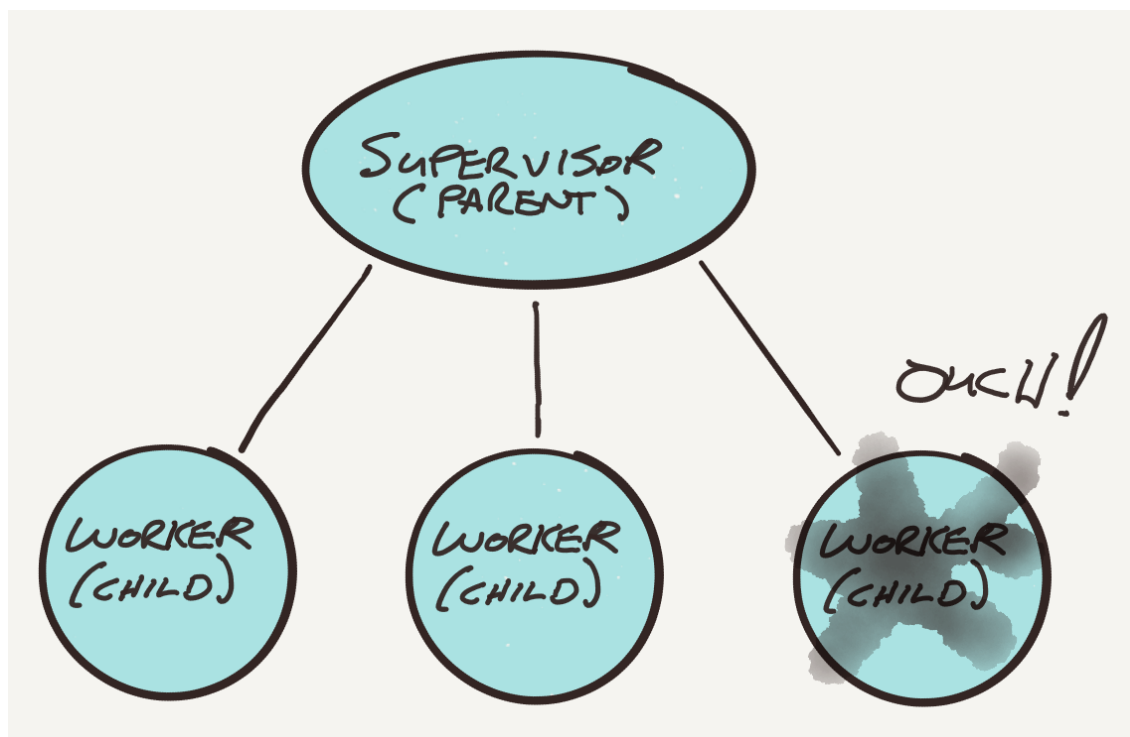


Figure 2-11. Worker actor has problem and notifies its supervisor

When the supervisor is notified it determines what should be done to handle the problem and how to get the worker actor back into a healthy state (see [Figure 2-12](#)).

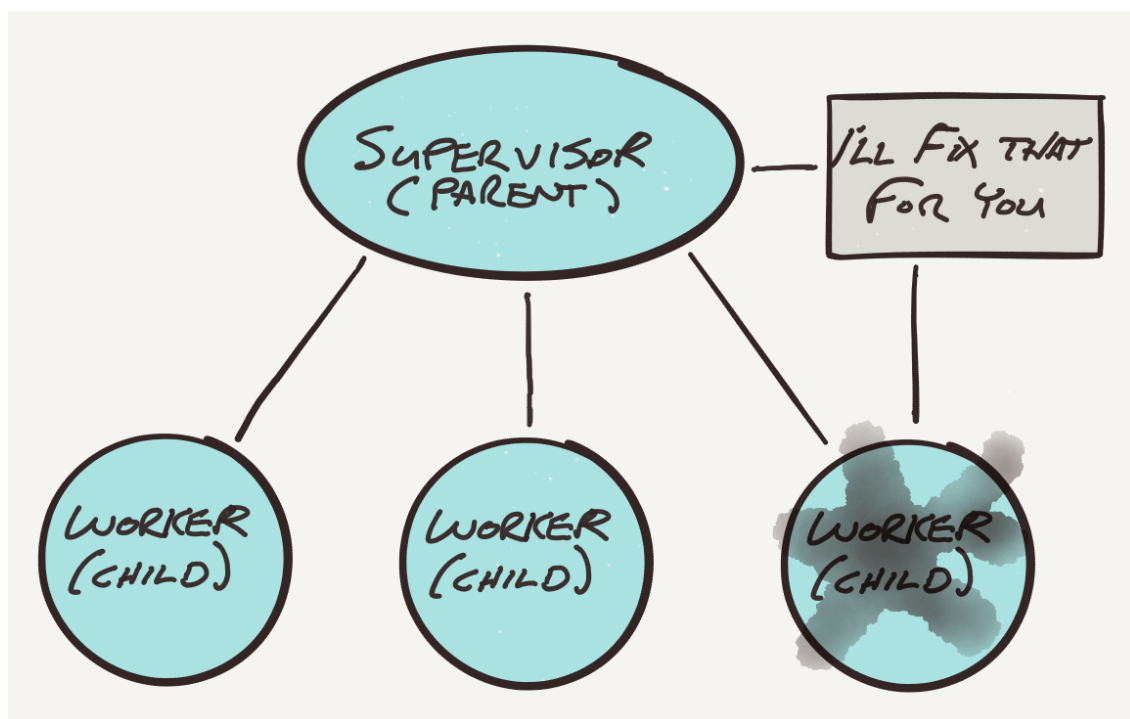


Figure 2-12. Supervisor fixes worker that has experienced a problem

Because the supervisor can send messages asynchronously, without having to wait for a response, the supervisor may send messages to each of its workers, which run independently and concurrently. This is in contrast to the traditional sequential programming model.

Let's consider a scenario where a supervisor actor receives a message to perform a task. In order for the supervisor to complete this request, it must complete three subtasks (see [Figure 2-13](#)):

- Obtain the customer profile
- Pull recent customer activity
- Calculate customer-specific sales opportunities

The supervisor delegates this work to three worker actors. The individual actors used are specifically designed to handle the individual tasks, so that one actor knows how to get a customer's profile and the other two actors are each designed to pull recent activity and calculate sales opportunities.

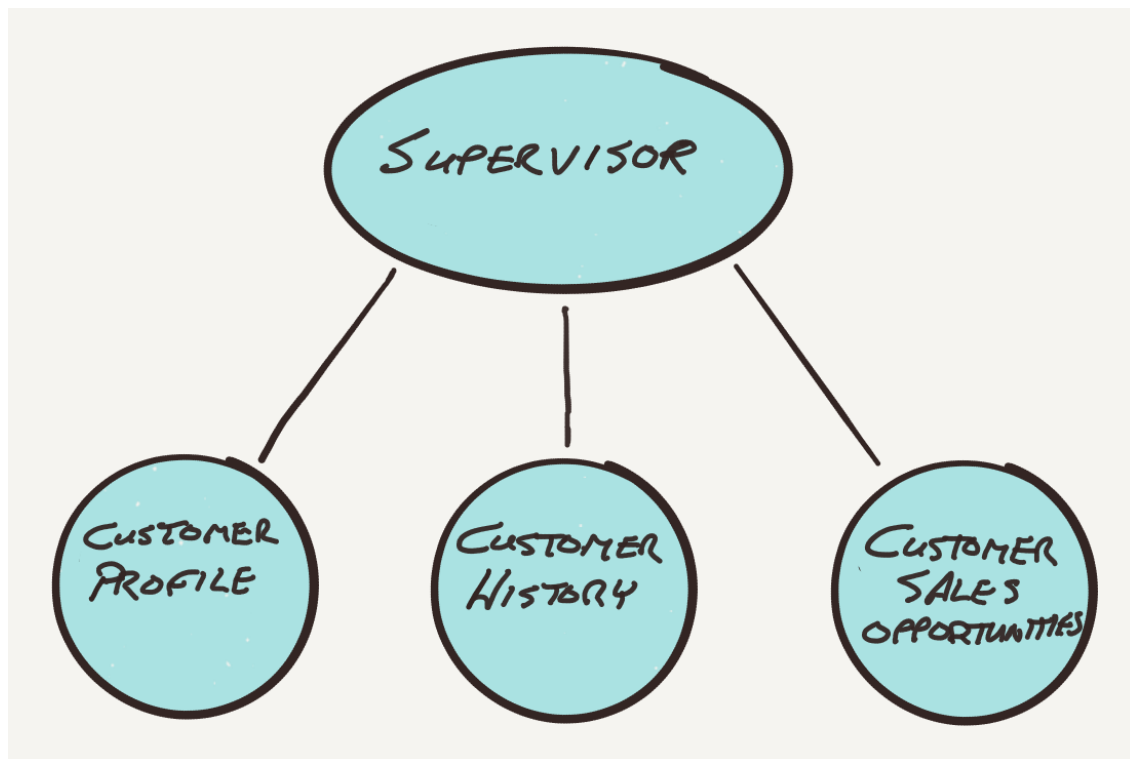


Figure 2-13. Supervisor actor delegates tasks to worker actors

The supervisor sends messages to each of the three worker actors asking them to perform their specific tasks. When each worker completes its task, it sends a message back to the supervisor with the results. Once all three workers have responded, the supervisor combines the results into a single response to the actor that sent it the initial request.

There are a number of benefits to this approach of delegating the work out to worker actors. One key benefit is *performance*. Because the workers run concurrently, the performance is based on the overall time it

takes for the supervisor to respond and reduced to the response time of the slowest worker.

Contrast this to the traditional synchronous approach: when these three tasks are processed sequentially, the overall response time = task 1 time + task 2 time + task 3 time (see [Figure 2-14](#)). In the Actor model, these three tasks are processed asynchronously, so that the overall response time = $\text{maximum}(\text{task 1 time}, \text{task 2 time}, \text{task 3 time})$.

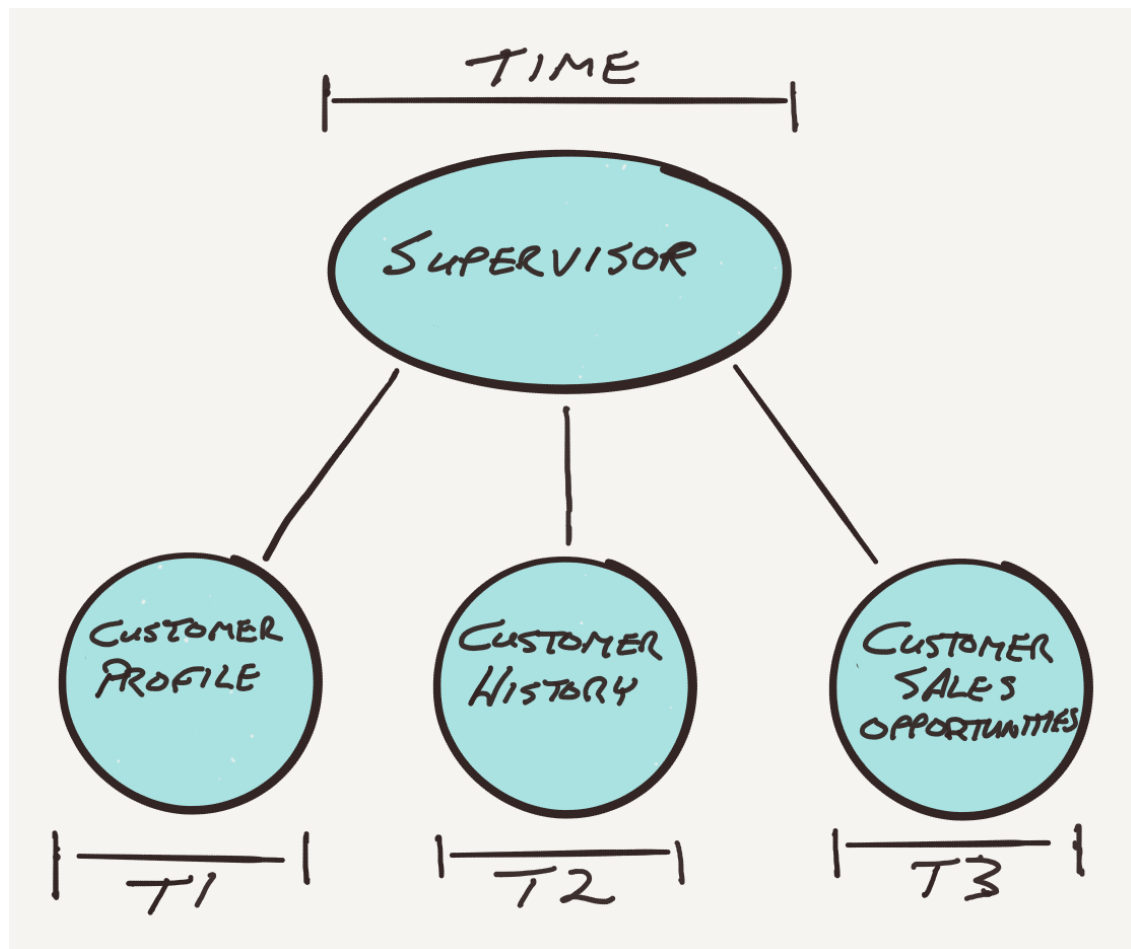


Figure 2-14. Worker actors perform tasks asynchronously

The asynchronous approach also *scales more efficiently* ([Figure 2-15](#)). The cost of adding more workers is much less than in a synchronous implementation.

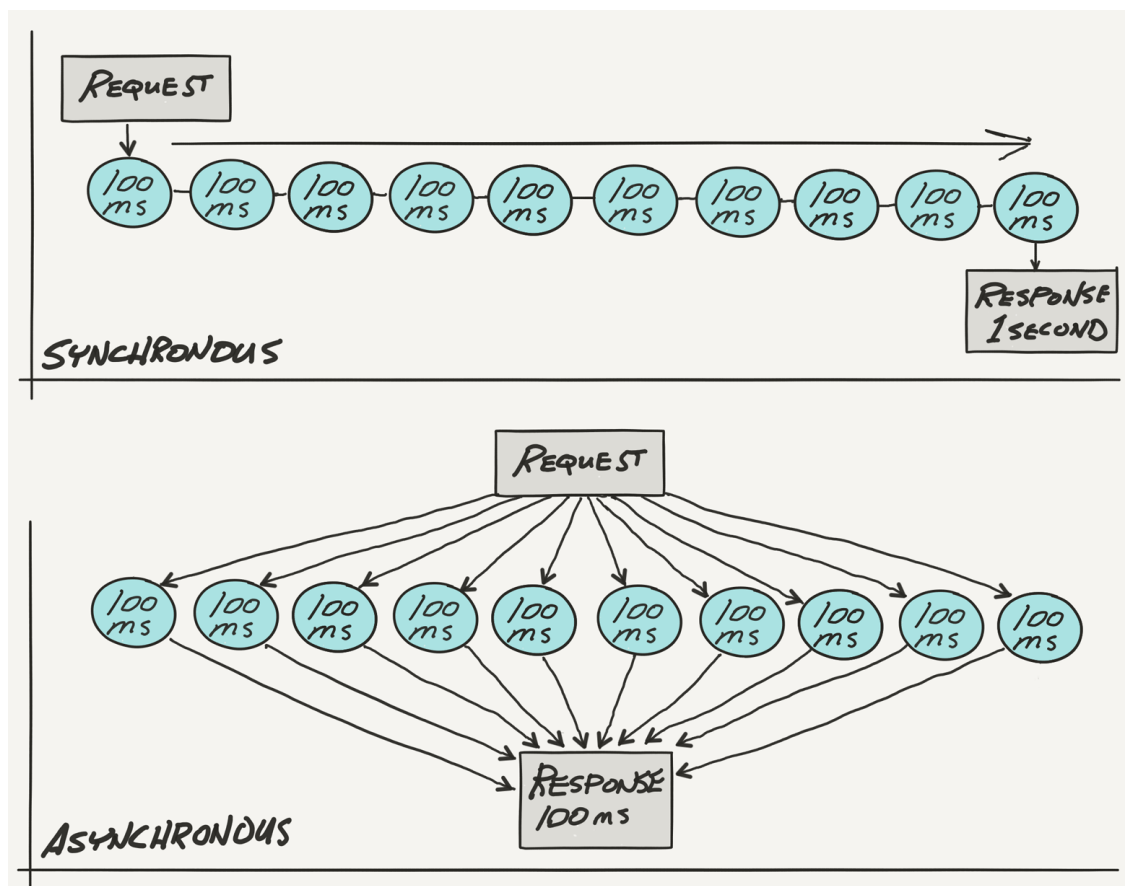


Figure 2-15. Ten 100 ms tasks performed synchronously take 1 second while the same ten tasks performed asynchronously take about 100 ms to complete

Say there is a need to add more functionality to this process, and you want to do more things for the customer to make the application more useful and attractive. With the asynchronous approach, the overall response time is still the response time of the slowest worker, while the response time for the synchronous approach increases with each added worker. This is exactly how highly efficient and feature-rich sites like Netflix and LinkedIn architect their systems.

Another benefit of the asynchronous delegated approach is related to *failure resilience* and *recovery*.

Recall the timeout approach previously discussed. In this scenario, the supervisor schedules a timeout message to be sent to itself when it sends tasks to each of the workers. This provides a way for the supervisor to handle workers that are unable or are slow to respond. The supervisor will only wait so long for the workers to respond. If a worker does not respond in time, then the supervisor decides how to proceed. How this is handled is application specific, but some typical approaches used are to simply omit the missing information, or to submit some default information in its place. The main point here is that a slow service or a down service is not going to break the system.

The main takeaways of this chapter are:

- Actors exchange messages asynchronously.
- Actors typically are set up to handle all contingencies, both when things work as expected and how to handle the unexpected.
- Actors can create other actors in a supervisor-worker relationship.
- Supervisors can delegate tasks to workers, and also take care of workers that run into problems.

[Support](#) [Sign Out](#)

©2022 O'REILLY MEDIA, INC. [TERMS OF SERVICE](#) [PRIVACY POLICY](#)